University of Pisa

Master Degree in Computer Science



Minimal imperative language with analysis and optimization and a minimal functional language with a type system

Compilation Techniques Project

Proff:
Giuseppe Lomurno, Roberta Gori, Andrea Corradini

Student:
Claudio Simonelli 714905

Academic Year 2025/2026

Chapter 1

Introduction

The goal of the project was to implement two compilers: one for a minimal imperative language called Miniimp, and another for a minimal functional language called Minifun. More specifically:

- For **Miniimp**: evaluation of the language, control and dataflow analysis, compiler optimizations and LLVM generation;
- For **Minifun**: evaluation of the language and a simple type system.

Chapter 2

How to run the code

The project was completely written in Java. It's divided in two indipendent packages: miniimp (compiler for the imperative language) and minifun (interpreter for the functional language). Both of those were designed to be execute by CLI reading the source code from external files to guarantee the maximum modularity.

2.0.1 System requirements

To execute the entire pipeline, you must have installed:

- **Java Development Kit (JDK):** At least version 17 for the pattern matching switch and text blocks support;
- **LLVM Framework:** Very important for the backend of miniimp. On Windows machines, it's also required a C/C++ environment (like Visual Studio Build Tools) for the linking of the standard library stdio.h.

2.0.2 Java's Code Compilation

Open the terminal in the main project directory (the one which contains the src directory) and execute these two following commands for the compiling of the two packages:

```
# Compilation of miniimp package
javac -d out src/miniimp/*.java
# Compilation of minifun package
javac -d out src/minifun/*.java
```

2.0.3 Miniimp: LLVM IR generation

Create a text file which contained the source code (for example: program.imp.txt) and execute the Java frontend with the following command:

```
java -cp out miniimp.CompilerMain program.imp.txt
```

The program will analyzes the code, apply the control flow graph optimizations and generates a program.ll file in the current directory.

LLVM Pipeline and Linking

Launch the following commands respectively for: the optimization of the IR (promoting the variables in SSA form), generation of the object file and creation of the native executable:

```
opt -passes=mem2reg program.ll -S -o program_opt.ll
llc -filetype=obj program_opt.ll -o program.o
clang wrapper.c program.o -o miniimp_app.exe
```

Execution of the binary file

Execute the just created binary file launching the following command:

```
.\miniimp_app
```

2.0.4 Minifun

Create a text file with the source code (for example: program.fun.txt) and execute the interpreter as follows:

```
java -cp out minifun.MiniFunMain program.fun.txt
```

Chapter 3

Implementation choices

3.1 Miniimp

3.1.1 Fragment 1

Task 1: Write a parser for the syntax

For the parsing I choose the recursive descent parsing technique because the grammar of Miniimp is ambiguous or too rigid for a deterministic parser. For example the expression not $\mathbf{x < 1}$ requires that the parser evaluates the operator $<$ first to avoid runtime errors. The parser implements this precedence stratifying the methods (parseBool calls parseBoolTerm).
I tried to ask AI to generate the parser, but looking and testing the code leads me to a critic bug: the ; symbol (as command separator) has the associativity property, so the if and the while commands consumed the body with the greedy method parseCmd ending up mistakenly swallowing subsequent commands outside the loop. I solved this issue by modifying the parser in a way that if and while commands accept a single atomic command (parseSingleCmd method). To include more commands in a branch or in a cycle, the user must use explicitly the grouping by () parentheses (<cmd>) as formally told by the grammar.

Task 2: Define a data type for the abstract syntax tree

Exploiting the modern Java 17+ features, I modeled the AST by an interface system and Java records for the internal nodes and the leafes. The nodes reflect the tripartition of the syntactic categories Prog, Cmd, Expr and BoolExpr.
I chose Java records because they guarantee native immutability, zero boilerplate code (getter, setter and so on) and extremely efficiency because of the native implementation of robust hash code functions at zero cost. In this way, I can completely exploit the positive sides of using hash based data structures like hash maps and hash sets. For the interfaces, I chose them because they allow me to define a close algebraic type system in a more safe way than the basic classes.

Task 3: Define data types and function for the runt-time environment

I implemented the memory state σ with a mutable class Memory which encapsulates a hash map from Strings to Integers and has two methods read and write. The read method search a value in the map and returns the value if and only if it's contained in it, otherwise It throws a deadlock runtime exception. The write method simply put a value in the map.

Task 4: Translate the deduction system into evaluation functions

I implemented a class Interpreter based on switch's pattern matching in Java. The interpreter has three distinct methods in perfect parallelism with the formal relations:

- **evalExpr:** $< \sigma, e > \rightarrow_e n$
- **evalBool:** $< \sigma, b > \rightarrow_b b$
- **evalCmd:** $< \sigma, c > \rightarrow_c \sigma^{'}$

I chose this kind of pattern matching because allows for near 1:1 mapping of the inference rules. In this way I can have a clear and short interpreter.

3.1.2 Fragment 5

Task 1: Define a data type and operations for CFGs

I create a class called CFG which encapsulates another static class called Node, which represent the single node of the graph. I implement this in this way because I think is very safe for the data saved in the single nodes. Every node has an unique id, a list of commands (because I chose maximal blocks), a list of outgoing edges, the input, output, gen and kill sets. The last four sets are implemented with a Map from String to Set<?>. The String represent the unique code for the dataflow analysis (DEFINED_VARIABLES, LIVE_VARIABLES, REACHED_DEFINITIONS). I chose Set over List because the dataflow analysis theory is based on set theory. In this way I will have algorithms that match 1:1 the mathematic equations of dataflow analysis. That's because the Java Collections Framework implements all the fundamental set theory operations in a efficient way. Talking about efficiency, Hash Sets in Java have the built in methods contains and add with $O(1)$ time complexity. In List contains that would be $O(n)$. The final reason for which I chose Hash Sets is for the fixpoint algorithm I had to write in the future dataflow analysis fragment. Choosing List would be critic because they are order sensitive (Hash Sets are not), so the check for a possibile reached fixpoint would have been much more difficult and inefficient to implement. I chose the wildcard ? for Sets because different analysis leads to different types of sets. Every node has a function for adding a new command to its list of commands and all the getter and setter methods for the four maps.
The CFG class also has an enumeration called EdgeType for the edge types that can be NORMAL for standard link between two blocks and TRUE or FALSE for the branches of if and while commands, a record which represent an edge (with two attributes: a node which represent the target node and an EdgeType), an entry node and an exit node.

Task 2: Define a function that given a MiniImp program AST returns its CFG

To solve this task, I implemented a class called CFGBuilder which relies on two main methods. The public method build takes the program's AST as input, allocates a new, empty Start node, and returns the final CFG by calling a recursive function named buildGraph. This function takes the body of the AST and the current node as arguments, recursively traversing the AST. Depending on the type of command encountered, it either appends the instruction to the current node or interrupts the linear flow to create new branches and nodes. Finally, it returns the updated terminal block to the caller, ensuring the correct concatenation of subsequent commands.

Other details

I chose maximal blocks over minimal blocks because:

- The entire CFG is more readable. This is a positive side I absolutely wanted in the testing phase;
- More difficult to build, but helps me in the writing of future algorithms based on CFG;
- Looking at the future dataflow analysis fragment, the visit of the CFG will be more efficient in the context of calculating the IN and OUT equations for the different analysis;
- The memory layout of the program will be more clean and will have less wasted memory.

Answering to your question **How do you generate the CFG for sequences?**, in the private buildGraph method mentioned before, I have a SeqCmd case where:

```
1    case SeqCmd seq -> {
2                CFG.Node outOfC1 = buildGraph(seq.left(), currentBlock);
3                yield buildGraph(seq.right(), outOfC1);
4            }
```

I recursively call the function passing the first command and the current node, saving the result in a temporary variable. In this way, the specific nature of the first command becomes irrelevant: thanks to the recursive computation, I directly obtain the exact exit node where the second part of the sequence must be attached. Finally, I return the result of the recursive call evaluated on the second command, using that temporary node as its starting point.

3.1.3 Fragment 6

Task 1: Extend CFGs to support annotations on blocks

Already explained in 3.1.2 task 1.

Task 2: Implement the support functions for the defined variables analysis

For the defined variables case, I don't need the kill set so I have only one support function to compute the gen set. To do this, I write a simple algorithm that works as follows:

- Start a BFS visit of the CFG;
- For each node of the CFG, visit all of its commands
- If the command is an assignment, then the variable is defined, so add it in the node's gen set

Task 3: Implement a function that given a CFG returns the CFG annotated with the defined variables

In this task I had to compute the fixpoint for the defined variables. The code starts computing two important informations: the universal set and the map of predecessors. I compute the universal set because the analysis is of "Must" type, which use the set intersection, so I needed to initialize the output sets to the neutral element of the set and in this case it's the set of all possible elements, to avoid that during the intersection all the sets become empty. The predecessors map allows me to easily visit backward (the predecessors are fundamental to the computing of the equations, so I want to access to them in the more easily way). The CFG natively exposes the outgoing edges, but this map allows, given a node, to look instanly which nodes points to it.
The starting node of the fixpoint cycle is the entry node of the CFG. This node always has:

- $In(b) = \{inp\}$
- $Out(b) = Gen(b) \cup Kill(b)$

For every other node, at the start they have $In(b) = \emptyset$ and $Out(b) = universalSet$.
The core of the algorithm is the fixpoint cycle. The idea is to compute the In and Out equations until at the end of an iteration I reach $In(b) = Out(b)$.

Task 4: Repeat the last two steps for live variables and reaching definitions

Live Variables:
I compute both gen and kill sets in one method. The idea is to visit che CFG starting with both sets empty. Next, I visit one at a time the commands of the block (starting from the first). In the case of assignment commands, I look at all the variables at the right of the assignment. If one variable is not yet in the kill set, then I add it to the gen set. After this, I look at the variable at the left of the assignment and I add it to the kill set (the variable at the left is fated to death). The if/while cases are similar with the difference in the fact that there are only readings, so I don't need to add any variable in the kill set.
To computing the fixpoint, I did a reverse visit of the CFG because the liveness informations goes from the bottom to the top of the program. Iterating the nodes in the reverse order means following the natural flow of the data. In this way, when I compute a node, its successor are probably already computed in the current iteration, reducing the total number of iterations needed to reach the fixpoint.
Because this is an analysis based on set union, at the start the input set is initialized at empty. The output set of the exit node is initialized with the output variable because at the end of the program the output variable is surely alive.

The fixpoint cycle compute each node computing the in and out equations. Given that this is a backward analysis, I first compute output set looking at successors, then I compute the input sets by looking at the current block itself.
In the end, I check if I reached the fixpoint by checking the condition only on input sets for performance reasons. I can do this because the output sets themselves are built by other input sets.
Reached Definitions:
Also in this case I compute both gen and kill sets in one method. The idea was to divide the algorithm in two passes:

- 1st pass: I collect all the program definitions in one big Map;
- 2nd pass: For each block, I check which variable it assigns and put into the gen set its definition. Then, I put into the kill set all the other definitions of those variables (taken from the map).

Since this analysis is based on set union, for each block I initialized the input and output set to empty, and since this is a forward analysis, I need a map of predecessors. For the fixpoint cycle, for each block I compute the input and output sets and do the stability check on the output sets because input sets are built from the output sets themselves.

3.1.4 Fragment 7

Task 1: Implement a function that given a CFG checks whether there are possibly undefined variables

The idea was iterate over all CFG blocks. For each block I create a "safe" set equals to the values of the input set. Then, I Iterate over all commands of the block and for all commands of type assign, if and while I take all the read variables using an auxiliary function. For each of those variables, if there aren't in my "safe set", I print an error. In the end, if the command is an assignment, I add the variable in the safe set because from this moment on, in that block, the variable is defined. The algorithm print a special error in the case of a possible use of an undefined variable. I didn't chose a custom or specific Expection because I didn't want to interrupt the flow of the program. The algorithm isolates the variable so I could print a very explicit warning on the standard error, reporting the anomaly without stopping the flow. I used the defined variables analysis to checking the undefined variables.

Task 2: Implement a compiler step for dead store elimination

The idea was starting from getting all the blocks of the graphs. For each block, I save the variables that are surely alive at the end of the block (its output set). I use a new HashSet because I don't want to modify the original data. I allocate a list where I put the commands who deserve to survive. I choose Linked List because in this way I will have an efficient inserting at first. After this, I cycle on the block commands starting from the last command (I do a reverse for loop). I do this because to know if an assignment is useless, I must first look at what happens after it. Assume that in the reverse for loop I meet an assignment x := y + 1 and the variable x is not contained in the current block output set, then nobody cares about x: the command is a dead store. Otherwise, x will be useful, so I need to save the command. So:

- I push the command on the top of the linked list. I do this because I'm reverse cycling the all commands and pushing on the top guarantee that the list has the original order;
- I remove x from the live variables;
- To compute x, the command needs to read y, so y becomes alive.

If i meet an if or a while, I can't eliminate anything because they control the flow of the program. I always save them on the top of the linked list, because the program always needs to read them to determine where to go. In the end of the algorithm, I simply substitute the current block list of commands with the optimized new list of commands, where all the useless assignments are deleted.

Task 3: Implement a compiler step for constant folding

I did folding by using pattern matching of java as follows:
Base cases are integer expressions and var expressions, they simply returns themselves.
The recursive case is binary operation. Assuming I have "$left\ op\ right$", I do:

- Recursive call on left
- Recursive call on right
- Check if both left and right sides are integer expressions. If yes, I take their values and do the actual operation. Otherwise, at least one of the two sides is not a number, so I can't do the actual operation and I return a new binary operation expression containing the partially folded branches to preserve the AST structure.

For the boolean expressions, the logic is analogous.

Task 4: Implement a compiler step for constant propagation

The algorithm iterates over all nodes of the CFG. For each node, I created a Map from Strings to integer expressions and a List of optimized commands both initialized at empty at the start of the iteration. The map allows me to track the constants in each node. Next, for each node I visit its commands.
If the command is an assignment, I take the expression at the left of the assignment and I call an auxiliary function (called propagateExpr) which takes and expression and a constants map and substitutes occurrences of known variables with their actual numerical values. Then, I take this propagate expression and I fold it. If the result of the fold is a number, I save it into the constants map. Otherwise, if the assignment is not constant, I remove the variable from the map because its value it's no longer predictable. In the end I create a new assignment instance with the original variable name and the folded expression at its right, and add it to the optimized commands list.
If the command is an if or a while I propagate and fold the boolean condition, create a new if or while instance with the folded condition and then add it to the list of the optimized commands.
In the end of iteration on the node, I simply substitute che original commands list to the optimized one.
Note that at the start of each maximal block the constants map is emptied. This guarantees maximum speed and total safety: by propagating the constants only within the linear and sequential flow of the single block, the risk of propagating incorrect values deriving from other control branches is deleted, without having to resort to complex global Data-Flow equations for the constants.

Task 5: Implement an optimization pipeline

I implement the optimization passes in a fixpoint loop. It runs constant propagation (which includes folding), recalculates Live Variables, and then runs Dead Store Elimination. It repeats until the CFG size stabilizes. I recalculated the Live Variables because propagation replaces many variable reads with integer constants. I chose this order because constant propagation (with folding) collapses the expressions eliminating the actual use of variables. This prepares the field for the recalculation of Live Variables, which detect as "dead" the variables no longer referenced. After this, the dead store eliminator use these updated informations to delete useless assignments. The entire process is incapsulated in a fixpoint loop because the optimization may leads to other possible optimizations like a new constants propagation.

Other details

The infrastructure of the optimizer has been designed with a Centralized Coordinator (the class Optimizer). I didn't want to fragment the logic in more classes. Single optimizations are implemented as independent methods which do the transformations in-place on the CFG. The Optimizer decides if the optimizations had an effect by using a technique called State Tracking based on the cardinality of the AST.

3.1.5 Fragment 8

Task 1 and 2: Implement a function that generates a LLVM IR file of the program and apply at least the mem2reg optimization with opt

I implemented a method in a class called LLVMGenerator which takes a CFG, the input variable and the output variable of the program. For every variable of the program, I generated an instruction "alloca i64" in the start block "entry". This creates a pointer in the stack memory. Every read of a variable becomes a load in a fresh register and every write becomes a store in the allocated pointer. There are no phi-nodes. I will give the code to the command "opt -passes=mem2reg", which transform these memory operations in SSA register and phi-nodes. During the CFG visit, every block is translated in a LLVM basic block with a label. At the end of every block, I generated a jump instruction:

- br label %target for unconditional jumps;
- br i1 %cond, label %true, label %false for if and while;
- ret i64 %val for terminate the function and return the output.

Task 3: Compile with the provided wrapper

See the subsection 2.0.3.

Other details

I handled the generation of fresh names by implementing a counter in the LLVMGenerator class. Every time I read a new variable or I met a new expression I called a function called nextReg to determine the new unique register associate with them. nextReg simply appends "%" at the incremented counter.
The choice over maximal blocks had a very positive impact on the generation of LLVM IR code. By translating maximal blocks, the .ll generated file has long linear sequence of instructions, reducing the code fragmentation. In particular, this approach has minimized the number of labels and almost nullified the unconditional jump instructions to link sequential instructions. So in the end I had a more compact source file and this implies It was more readable for humans.
I chose to implement the alloca/load/store paradigm because computing the phi-nodes manually would have required the complex construction of the dominance frontiers. By allocating mutable variables on the stack, I deliberately delegated the SSA conversion to the LLVM optimizer, relying on the mem2reg pass to automatically promote memory allocations to registers and insert the correct phi-nodes.

3.2 Minifun

3.2.1 Fragment 2

Task 1: Write a parser for the syntax

For continuity and simplicity, also Minifun has a recursive descent parser. Also that parser is written with the help of the AI.

Task 2: Define a data type for the abstract syntax tree

For the same reasons I explained for Miniimp, I chose interfaces and records also for the AST of Minifun.

Task 3: Define data types and function for the run-time environment

I implemented the environment of Minifun with a class called Environment, designed to support the strict immutability constraint of functional languages. Unlike the Miniimp's memory, I define the environment of Minifun as a final (to guarantee immutability) map from Strings to actual values. I implemented these values with an interface called Value, defined with four record attributes: one for integers, one for booleans, one for the closures and one for recursive closures.
The Environment class has two constructors, that's because I needed a constructor to create a brand new

empty environment and another to clone the current environment, which is a very useful operations when we want to extend the environment ($p[x \to v]$). The first constructor is public and the second is private due to security reasons (I don't want the other classes to access the second constructor).
Then, the class has two public methods: read and extend. Read search for a value in the environment map and return it if it's contained, throwing an exception otherwise. Extend is the transposition of $p[x \to v]$, It takes a new variable with its value and return a new environment initialized with the old one and then extended with the new variable added.

Task 4: Translate the deduction system into evaluation functions

To manage the Minifun functions, I create a ClosureValue object which capture the the parameter, the function body and the actual state of the environment in that moment. The main difficult was managing the recursive function case. To call itself, a recursive function must be contained in his own closure. However, implement this in a classic way by attempting to inject the function into the environment while the closure is being built would have violated the strict immutability constraint and it would bring to infinite circular references in memory. So, I introduced a dedicated Value called RecClosureValue. In the middle of definition (LetFunExpr case), the closure doesn't capture itself but it memorizes only its name. The ring is closed in a lazy way only in the moment of the application (AppExpr case): an instant before the execution of the function body, the Evaluator class (the one which is responsible for evaluating the expressions) dinamically extend the environment by adding in it the function itself. This architecture implements the semantic rule of RecFunApp maintaining the immutability.

3.2.2 Fragment 3

Task 1 and 2: Complete the type system of MiniFun with rules for handling the annotations

To support the static analysis, I extended Minifun with a Type interface and three records that implement it: IntType, BoolType, FunType and I create a class called TypeEnvironment with a final hash map attribute from Strings to Types. Then, I implemented a class called TypeChecker that has a method called check that takes an expression and the type environment and return the type of the expression. The method translates directly the slides inference rules: Num, BoolT, BoolF, Var, Plus, ITE, Fun, FunApp, Let, LetFun.

Task 3: Extend the syntax of MiniFun with annotations

To avoid the need of a complex type inference algorithm, I extend the syntax and the AST of minifun by modifying the function declaration construct from $fun\ x => t$ to $fun\ x : T => t$ forcing the user to specify the parameter type, and by modifying the recursion construct requesting the entire function signature. As we can see in the MiniFun class, the FunExpr and LetFunExpr record now has a Type attribute too.

Task 4: Define data types for types and environments

To model the type domain, I create an interface Type from which derives the concrete types (records) IntType, BoolType and FunType. FunType has two attributes: Type left and Type right which are respectively the domain and the codomain. In parallel to the Environment, I implemented the class TypeEnvironment. This class maps the variables names (String) to their static types (Type). As the Environment class, at runtime TypeEnvironment follows a clone and extension logic (with the method extend, practically with the same behaviour of the Environment's extend) to guarantee that the lexical scoping is respected during the static analysis too.

Task 5: Translate the deduction system into a typechecking function that either produces the type of a term or errors out

I translated the deduction system in the class TypeChecker. This class has a method check which takes an expression and the type environment and returns the type of the expression. The method navigate the AST using the pattern matching. For each node, the method recursively checks the subtree types and verifies if they respect the formal rules conditions. I choose a recursive approach for a more intuitive and readable algorithm.

Task 6: Bonus points if the error explains why it fails to typecheck

To satisfy this requirements, I designed the type checking system to not returns generic errors like NullPointerException. In the case when the type checker meets a type inconsistency between the expected type and the computed one, it throws a personalized Exception called TypeException.